

EcoEvoSim: a general-purpose package for adaptive dynamics-style simulations

György Barabás^{1*}, Gaurav Baruah², Anuraag Bukkuri³

¹Linköping University, SE-58183 Linköping, Sweden

²Bielefeld University, D-33501 Bielefeld, Germany

³City St George's, University of London, EC1V 0HB London, United Kingdom

*Corresponding author (email: gyorgy.barabas@liu.se)

Abstract We introduce `EcoEvoSim`, a Julia package for simulating eco-evolutionary dynamics. The package takes an ecological model together with the evolvable trait(s) and automates the process of iterating between solving these equations and introducing new, random mutant phenotypes into the system. `EcoEvoSim` comes with the companion package `ecoevosimr` that provides a fully functional interface for R users. The package works with virtually any demographically deterministic ecological model expressible via a set of ordinary differential or difference equations: multidimensional trait spaces, auxiliary non-evolving variables (such as resources), and age-, stage-, or spatially-structured populations are all supported. We demonstrate the use of the package through a competition-proliferation tradeoff example that we interpret in the context of tumor diversification: tumor clones either proliferate rapidly, or are efficient resource users and therefore locally competitively dominant. By uncovering the ecological structure that arises from the simulated eco-evolutionary process, we also highlight how the properties of this structure can be exploited to efficiently dismantle the tumor community.

1 Introduction

Eco-evolutionary dynamics (Govaert et al. 2019; Klausmeier et al. 2020) arise via the reciprocal coupling between ecological interactions among populations and the evolutionary change of their traits. It can (and has) been conceptualized in a wide variety of ways, each method having its advantages and drawbacks. **Quantitative genetic models**, for example, assume a simplified underlying **genetic architecture determining the intraspecific distribution of traits** (Barton et al. 2017). This framework is especially useful when ecological and evolutionary processes happen on comparable time scales, yet we still wish to retain some **analytical tractability even at the community level** (Slatkin 1979; 1980; Taper and Case 1985; Schreiber et al. 2011; Vasseur et al. 2011; Barabás and D'Andrea 2016; Mullon and Lehmann 2019; Pastore et al. 2021; Barabás et al. 2022; Baruah et al. 2025). On the downside, there is no easy way of representing speciation in this framework. By contrast, **adaptive dynamics** (Dieckmann and Law 1996; Metz et al. 1996; Geritz et al. 1998; Brännström et al. 2013; Metz and Geritz 2016) yields **a purely phenotypic description of trait evolution, setting all genetic details aside**. In its standard formulation, it also assumes a strict time scale separation between ecological and evolutionary dynamics, so that ecological interactions equilibrate between successive mutation events. In return, these assumptions make it feasible to describe the process of speciation—so in this sense, quantitative genetics and adaptive dynamics have complementary strengths and weaknesses.

Time scale separation is an abstraction, and has been recognized as one for a remarkably long time (Levins 1968; Roughgarden 1979). While restrictive, it is not built into the basic algorithm of adaptive dynamics itself: the underlying loop of “integrate the ecological dynamics, introduce a mutant, prune extinct phenotypes, repeat” is sensible even when mutations are frequent enough that several may appear simultaneously, making ecological and evolutionary time scales overlap. While this reduces (but does not eliminate; see, e.g., Mészéna et al. (2005)) analytical tractability, this is not a concern when considering simulated evolutionary trajectories. Adaptive dynamics-style evolutionary simulations therefore have an important place in studying eco-evolutionary processes.

In practice, applying this approach to a new model has typically meant one of two things: deriving invasion fitness expressions analytically for the small class of models in which this is tractable, or writing a bespoke simulator from scratch for everything else. The result is that the same machinery—an outer loop over mutation events wrapping an inner loop of ordinary differential equation (ODE) integration, **mutant introduction**, and **extinction pruning**—is repeatedly reimplemented, in a way that is difficult to extend across model classes. What is missing is a **simulation** tool that can do **three things at once**. First, it must integrate the **ecological dynamics** of an arbitrary, user-specified model. Second, it must overlay an evolutionary process that **gradually mutates** the traits of the resident populations and asks whether each new mutant can establish. Third, it must do so in a way that does **not commit the user to any particular ecological model**: **stage-structured, multitrait, resource-explicit** settings should all be expressible without rewriting the simulator, both in discrete and continuous time. On top of these desiderata, the tool should be efficient and easy to use.

Several existing simulation frameworks occupy parts of this space, each filling a distinct niche. At one end of the spectrum lies a large class of **individual-based simulators**, of which **Avida** (Ofria and Wilke 2004; Dolson and Ofria 2021) is a prominent example: these platforms track every individual organism explicitly, capturing demographic stochasticity and the full granularity of mutation and selection. **Gillespie eco-evolutionary models** (Dalui and DeLong

2025) take a related stochastic approach, using the Gillespie algorithm to simulate birth-death events and thereby track multi-trait, multi-species dynamics under nonequilibrium, demographically stochastic conditions. At the other end of the methodological spectrum, Christopher Klausmeier’s EcoEvo Mathematica package (Klausmeier 2024) is tailored to symbolic adaptive dynamics calculations, using computer algebra to derive invasion fitness expressions and pairwise invasibility plots analytically. A fourth class of tools, exemplified by gen3sis (Hagen et al. 2021), targets macroecological and macroevolutionary questions: gen3sis is a spatially explicit R engine designed to reproduce large-scale biodiversity patterns (species ranges, phylogenies, latitudinal diversity gradients) emerging from dispersal, speciation, and trait evolution over geological timescales.

Here we introduce EcoEvoSim, a Julia package occupying a middle ground between analytically tractable and individual-based frameworks. Like Klausmeier’s package, EcoEvoSim is grounded in user-specified analytical equations for the ecological dynamics, but like Gillespie eco-evolutionary models, it simulates evolutionary trajectories numerically rather than deriving them symbolically. Unlike both individual-based simulators and gen3sis, it is designed for community-level adaptive dynamics questions at the scale of interacting populations rather than individual organisms or biogeographic regions. The Julia programming language (Bezanson et al. 2017) is well suited to the task: it offers high performance, a sophisticated multiple-dispatch type system, and the mature DifferentialEquations.jl ecosystem (Rackauckas and Nie 2017) for ODE integration. EcoEvoSim builds on this base. Once the user defines an ecological model and the evolvable traits, the package automatically iterates ecological integration and rare-mutant introduction for as many evolutionary steps as the user wishes. Recognizing that most practicing biologists work in R rather than Julia, we additionally provide ecoevosimr, a lightweight R wrapper that exposes the same functionality as native R functions, returns simulation output as tibbles ready for analysis with the tidyverse (Wickham et al. 2019) and ggplot2 (Wickham 2016), and requires no Julia knowledge beyond the ability to write one-line equations as a string.

To showcase the package on a concrete biological problem, we apply it to tumor dynamics. Tumors are particularly natural systems for adaptive dynamics: the standard idealizations of asexual reproduction and clonal inheritance, which elsewhere are useful but rough approximations, hold essentially literally for cancer cell populations. A tumor is itself a diverse community of genetically and phenotypically distinct clones (Nowell 1976; Marusyk et al. 2012; McGranahan and Swanton 2017), and this intratumoral heterogeneity is one of the principal obstacles to durable treatment: the more diverse the tumor, the more likely it is to harbor clones that are intrinsically resistant to any given therapy, or that can rapidly evolve resistance once treatment is applied (Greaves and Maley 2012). Standard strategies that target whichever clone is currently dominant often only release competitive pressure on resistant variants and accelerate their rise (Gatenby and Gillies 2008; Gatenby et al. 2009). Community ecology offers a complementary perspective: the clones in a tumor are members of an ecological community whose composition is shaped by competition for resources, predation by the immune system, and the spatial heterogeneity imposed by the vasculature (Merlo et al. 2006; Aktipis and Nesse 2013; Korolev et al. 2014), and a central insight from community ecology is that such diversity is structured by tradeoffs (MacArthur and Levins 1967; Chesson and Huntly 1997; Pásztor et al. 2016). If intratumoral heterogeneity reflects ecological structuring, then the rules that *maintain* diversity in tumors can in principle be exploited to *dismantle*

it: by perturbing the right part of the community, indirect ecological effects can be made to collapse much of the tumor at once, without ever directly attacking the most resistant cells.

This work has two purposes. The first is to describe the `EcoEvoSim` package: its conceptual framework, core types, design based on functional programming patterns that allows the same engine to drive a wide variety of different model types, and its R interface (Section 2). Second, we apply the package to the cancer scenario sketched out above: a competition-proliferation tradeoff model (Adler and Mosquera 2000; D'Andrea et al. 2013; Liao et al. 2022; Miller and Max 2025) in which proliferation capacity is the evolving trait. We show that the simulation autonomously generates a stable, hierarchically organized community of coexisting clones, that its community matrix exhibits the qualitative structure expected of hierarchical competition, and that the matrix's inverse reveals strong cascading effects. We close by sketching the therapeutic implications under the heading of *ecological cascade therapy* (Section 3).

2 Package description

2.1 Conceptual framework and core types

`EcoEvoSim` implements eco-evolutionary simulations by receiving a user-specified ecological dynamical system specification, and then alternating between two phases. In the *ecological phase*, the ecological community's dynamics are projected forward in time, either for a fixed amount of time or until reaching steady state. Since the package only allows for continuous-density models without demographic stochasticity, populations whose density falls below a specified extinction threshold are removed after this step. In the *evolutionary phase*, a rare mutant phenotype is introduced into the equilibrated and pruned community. The mutant is created by taking the phenotype of an existing resident, perturbing its trait value by a small random variate, and introducing this new phenotype into the community at a low population density (specified by the user). Iterating these two phases many times produces a complete eco-evolutionary trajectory, capturing both directional evolution along selection gradients and evolutionary branching (Geritz et al. 1998; Meszéna 2005).

The package is organized around five custom types that are used to represent populations and communities. All of these are parameterized via a numeric type parameter `T` through which population densities and trait values are represented. One can usefully think of `T` as a double-precision floating-point value (`Float64`), but the main advantage of Julia's type system is precisely that one need not commit to a single specific type. If additional numerical accuracy is required, one may instead define population sizes and trait values as `Float128`, or even as arbitrary precision `BigFloat` values. The basic types, in order from more elementary to higher-level ones that are composed from those elementary types, are:

- `PopulationSize{T}` wraps a vector of densities. For unstructured populations the vector has a single element; for stage- or patch-structured populations it has one element per class.
- `Phenotype{T}` wraps a vector of trait values. A scalar trait is represented as a length-1 vector; a multidimensional trait space simply uses a longer vector.
- `Species{T}` pairs a `PopulationSize` with a `Phenotype`. It represents a single species or clone, defined through its density (in each stage class / patch) and trait vector.
- `Community{T,AuxClasses}` groups three things: a vector of `Species`, an optional vector of auxiliary `PopulationSizes` (representing parts of the system that have ecological dynamics but do not undergo evolution, such as resource pools), and a `time` field tracking the elapsed

136 ecological time. The type parameter `AuxClasses` records the number of auxiliary variables,
137 allowing the compiler to specialize on the system structure.

- 138 • `EvoHistory{T,AuxClasses}` is simply a vector of `Community` objects, representing snap-
139 shots along an evolutionary trajectory. Each entry corresponds to the community state at
140 one particular mutation event, and is what the function `evolve` returns.

141 These types are designed so that the same simulator handles a Lotka–Volterra community
142 of unstructured populations and stage-structured models with explicit resources, without
143 any need for tracking special cases. Whenever the engine needs to talk to the differential
144 or difference equation solver, the community is reshaped into a flat state vector by a func-
145 tion `unpackCommunity`; after integration, `packCommunity` reassembles the result into a new
146 `Community`. These operations happen automatically, so an end user need not be aware of them.

147 2.2 Configuration

148 A simulation is fully specified by a single value of type `EcoEvoConfig`. This custom type
149 bundles four things: an ecological model (`ecoDyn`), a method for specifying how new mutant
150 phenotypes are generated from already existing ones (`mutationGenerator`), parameters sent
151 to the Julia equation solver (`integrationParams`), and an extinction threshold population
152 density (`extThreshold`) below which a phenotype is pruned from the community at the end
153 of each ecological phase. Here is an example implementation:

```
154 config = EcoEvoConfig(  
155     ecoDyn = <ecological model>,  
156     mutationGenerator = <mutant-generating closure>,  
157     integrationParams = IntegrationParams(  
158         maxTime = 1e12,  
159         algorithm = Rodas5(),  
160         abstol = 1e-8,  
161         reltol = 1e-8  
162     ),  
163     extThreshold = 0.003  
164 )
```

165 The first two fields, `ecoDyn` and `mutationGenerator`, are higher-order factory functions; i.e.,
166 functions whose output is itself a function. While this is good to know for understanding
167 the internal organization of `EcoEvoSim`, in practice there are both built-in solutions as well
168 as helper functions which hide these details from the user, making their implementation
169 straightforward (see below).

170 The `integrationParams` field is forwarded to `DifferentialEquations.jl` (Rackauckas
171 and Nie 2017). This package supports solving both discrete- and continuous-time systems.
172 The first argument to `IntegrationParams` is `maxTime`, which is the number of time units (in
173 continuous time) or time steps (in discrete time) for which the dynamics should be integrated
174 or iterated. The second argument, `algorithm`, can be any standard integration method such as
175 `Rodas5()` or `Tsit5()` for differential equations, or it can be `FunctionMap()` to iterate recursion
176 equations. It is also possible to project the dynamics forward until steady state is reached,
177 using `DynamicSS()` for ODEs and `DiscreteSS()` for recursions; in that case, `maxTime` can be
178 specified as `Inf`. The absolute and relative tolerances, for integration or for judging whether
179 steady state has been reached, are controlled by `abstol` and `reltol`. Briefly, the smaller

the absolute tolerance, the more accurate results are near zero, and the smaller the relative tolerance, the more accurate results are near one.

2.3 The driver loop

Given a configuration `config` and a starting community, the function `evolve(community, config, nMutations)` returns an `EvoHistory` (vector of communities). Its inner loop, exposed as `singleEvoStep`, has only three substantive steps:

1. Solve the ecological system for `maxTime` units (or to a steady state) using `ecoDyn(community, config)`.
2. Remove any species with density below `extThreshold` (with the function `removeExtinct`).
3. Generate a new mutant via `config.mutationGenerator(community)` and append it to the community.

The first input to `evolve` can either have the type `Community` (in which case it acts as the initial community from which the evolutionary history is built) or `EvoHistory` (which simply appends the additional mutational steps to an existing history). The latter option means that evolutionary simulations can be run incrementally, resuming them with no boilerplate.

2.4 Specifying ecological models

The most distinctive feature of the package is how ecological models are specified. The field `EcoEvoConfig.ecoDyn` does not hold an ODE function directly. Instead, it holds a function whose output is itself a function (a higher-order factory function). In particular, given a `Community`, the result is an ODE or discrete-time function $(u, p, t) \rightarrow du$ specialized to that community. This indirection is what allows the same configuration to handle communities of different size: each time a mutant is added, a fresh function is generated whose closures capture the new species count, the new vector of traits, and (optionally) any auxiliary variables such as resources.

While this is beneficial for the design of the system, the goal is to hide these details from the end user, who should only worry about specifying the model equations and not about having to hand-craft factory functions. The system contains three helpers which, given the equations, will automatically lift them to being the required higher-order functions; they are intended to cover the vast majority of possible use cases.

2.4.a The `lotkaVolterra(growthFn, kernelFn)` helper

When traits are explicitly notated, the generalized Lotka–Volterra model (Case 2000) reads

$$\frac{dn_i}{dt} = n_i \left(b(z_i) + \sum_{k=1}^S a(z_i, z_k) n_k \right), \quad (1)$$

where n_i is the population density of phenotype z_i , $b(z_i)$ is its intrinsic growth rate, S is the total number of phenotypes in the system, and $a(z_i, z_k)$ is the per capita effect of phenotype z_k on phenotype z_i . In `lotkaVolterra(growthFn, kernelFn)`, the user supplies these as Julia functions, with `growthFn` representing $b(z_i)$ and `kernelFn` representing $a(z_i, z_k)$. Since for any given community with fixed species the vector of intrinsic rates and the matrix of interaction coefficients remain constant, they are computed just once at the start of every ecological phase instead of being recomputed for every step of the ODE solver. In addition, `lotkaVolterra` benefits from a hand-crafted function for computing the Jacobian matrix for Eqn 1. The Jacobian J_{ij} is easy to compute analytically:

$$J_{ij} = \frac{\partial(dn_i/dt)}{\partial n_j} = \frac{\partial}{\partial n_j} \left[n_i \left(b(z_i) + \sum_k a(z_i, z_k) n_k \right) \right] \quad (2)$$

$$= \delta_{ij} \left(b(z_i) + \sum_k a(z_i, z_k) n_k \right) + n_i a(z_i, z_j),$$

where δ_{ij} is the Kronecker symbol (equal to 1 if $i = j$ and to 0 otherwise). Implementing this Jacobian as a function and supplying it to the solver obviates the need for the algorithm to evaluate the Jacobian numerically at every integration step, resulting in much faster execution speeds.

To give a very simple example, suppose that there is a single trait z measuring environmental adaptation. A value of $z = 0$ is the best-adapted phenotype. This is expressed by choosing $b(z) = 1 - z^2$, a growth function whose maximum is at zero. In turn, we assume that there is only a single resource in the system and every phenotype competes equally for it, so $a(z_i, z_j) = -\alpha$ is a constant. A Lotka–Volterra model with these ingredient functions can be defined as follows:

```
alpha = 1

b(z) = 1 - sum(z)^2
a(zi, zj) = -alpha

ecology = lotkaVolterra(b, a)
```

The summation over z in $b(z)$ is there because the package treats z as a vector, even when it has only a single entry like here. Using `sum` converts this single-entry trait vector to a simple value. Alternatively the definition `b(z) = 1 - z[1]^2` would work equally well.

2.4.b The `unstructuredModel(eqnFn; precompute, auxDynamics)` helper

This helper function is designed for any model **without** age-, stage-, or spatial structure. It allows the user to write the dynamical equations directly. For example, using the same `b(z)` and `a(zi, zj)` functions as above, implementing the generalized Lotka–Volterra model of Eqn 1 reads

```
alpha = 1

b(z) = 1 - sum(z)^2
a(zi, zj) = -alpha

ecology = unstructuredModel() do i, n, z, aux, S, t
    n[i] * (b(z[i]) + sum(a(z[i], z[j]) * n[j] for j in 1:S))
end
```

The closure arguments of `unstructuredModel` are purely positional, i.e., their meaning is fixed by the order in which they appear and not by the symbols the user chooses to represent them. The arguments are, in order: species index, population vector, trait vector, auxiliary variable vector, number of species, and—optionally—time. The user may name them anything; here we used `i`, `n`, `z`, `aux`, `S`, and `t`, respectively. The presence or absence of the final time argument is detected automatically by counting how many arguments the closure accepts. Including it

activates time-explicit dynamics, while omitting it yields an autonomous system (so in the example above, it can be safely left out). The closure returns dn_i/dt in continuous time, or $n_i(t+1)$ in discrete time.

An optional `precompute` keyword takes a function $(z, S) \rightarrow \text{pre}$ that is called once at the start of each ecological phase, and whose result is forwarded to the equation function as an extra argument. This can be useful if we want to prevent the system from evaluating quantities at every integration step that we know stay constant. The generalized Lotka–Volterra model is a case in point: its implementation via `unstructuredModel` above is inefficient, because the algorithm does not know that $b(z[i])$ and $a(z[i], z[j])$ need not be evaluated at every iteration of the integrator. The `precompute` helper resolves this inefficiency:

```
ecology = unstructuredModel(
  precompute = (z, S) -> (
    b = [b(z[i]) for i in 1:S],
    a = [a(z[i], z[j]) for i in 1:S, j in 1:S]
  )
) do i, n, z, aux, S, pre
  n[i] * (pre.b[i] + sum(pre.a[i, j] * n[j] for j in 1:S))
end
```

(There is no danger of a naming conflict: $b(z)$ and $\text{pre}.b(z)$ are kept separate, and the same for a .) This forces trait-dependent quantities that are constant between mutation events to be computed exactly once per ecological phase, rather than at every ODE step. The resulting gain in speed can be substantial, especially for communities consisting of many phenotypes.

An optional `auxDynamics` keyword accepts a function $(\text{aux}, n, z, S) \rightarrow \text{Vector}$ that returns the time derivatives of the auxiliary variables, enabling models with explicit non-evolving quantities such as resources or abiotic variables. The auxiliary variable vector `aux` is forwarded to `eqnFn` so that species dynamics can depend on the current values of these quantities. The same time-detection rule applies: appending `t` as a final argument activates time-explicit resource dynamics. When `precompute` is also provided, the precomputed object precedes `t` (if present): `auxDynamics(aux, n, z, S, pre)` or, with explicit time-dependence, `auxDynamics(aux, n, z, S, pre, t)`.

2.4.c The `structuredModel(eqnFn; precompute, auxDynamics)` helper

Communities of populations structured by age, stage, or spatial patches are handled by the analogous `structuredModel`. The equation function signature gains a stage class / patch index k , becoming `eqnFn(i, k, n, z, aux, S, K)`, where $n[i, k]$ is the density of species i in patch / stage k , S is the species count, and K is the number of patches / stage classes. The `auxDynamics` and `precompute` keywords work as described above for `unstructuredModel`, with the differences that `precompute` takes $(z, S, K) \rightarrow \text{pre}$ and `auxDynamics` takes $(\text{aux}, n, z, S, K) \rightarrow \text{Vector}$.

As an illustration, we implement the two-patch model of Mesz na et al. (1997) and Szil gyi and Mesz na (2009). This model is a simple extension of the one we have been using as an example. The difference is that the two patches have different environmental conditions: the optimal phenotype is y_k in patch k , so $b_k(z_i) = 1 - (z_i - y_k)^2$ is the patch-specific intrinsic growth rate function. Local competition in a patch is modeled as before: $a(z_i, z_j) = -\alpha$, so the patch-specific density dependent mortality term is $-\alpha \sum_i n_{i,k}$, where

302 $n_{i,k}$ is species i 's population density in patch k . The growth plus density-dependent term sum
 303 to $g_{i,k} = 1 - (z_i - y_k)^2 - \alpha \sum_i n_{i,k}$, defining the local dynamics in each patch. Additionally,
 304 there is symmetric dispersal across the patches at a rate μ . Putting these ingredients together,
 305 the governing equations read

$$\frac{d}{dt} \begin{pmatrix} n_{i,1} \\ n_{i,2} \end{pmatrix} = \begin{pmatrix} g_{i,1} - \mu & \mu \\ \mu & g_{i,2} - \mu \end{pmatrix} \begin{pmatrix} n_{i,1} \\ n_{i,2} \end{pmatrix}. \quad (3)$$

306 Translating Eqn 3 directly into a `structuredModel` gives:

```
307 using EcoEvoSim
308
309 mu = 0.1
310 alpha = 1.0
311 y = [-1/2, 1/2]
312
313 ecology = structuredModel() do i, k, n, z, aux, S, K
314     localGrowth = 1 - (z[i][1] - y[k])^2 - alpha * sum(n[i,k] for i in 1:S)
315     (localGrowth - mu) * n[i,k] + mu * sum(n[i,l] for l in 1:K if l != k)
316 end
```

317 2.5 Factory functions for mutation generation

318 Mutant generation follows the same pattern as defining the ecological dynamics, requiring
 319 a higher-order factory function whose output is itself a function. The output
 320 function's type is `Community -> Community`. When applying this generated function
 321 to a community, we get back another community with a new, mutant phenotype
 322 added. There are four basic built-in functions which already realize this pattern; they
 323 are called `generateMutant`, `generateMutantWeighted` and `generateMutantSpatial` and
 324 `generateMutantSpatialWeighted`. All of them follow the same structure:

```
325 generateMutant(invaderPopsiz = 0.001, variance = 0.002^2)
```

326 creates a function that, when applied to a community, picks a parent species at random,
 327 perturbs its trait by a draw from a normal distribution with mean zero and covariance
 328 matrix Σ (which may also be supplied as a scalar, in which case it is interpreted as
 329 the multiplier of the identity matrix), and appends the resulting mutant at the prescribed
 330 invader density, `invaderPopsiz`. The variant `generateMutantWeighted` is the same except
 331 the parent species is picked with a probability proportional to its density. This is more
 332 realistic if mutation rates are interpreted per individual. The `generateMutantSpatial` and
 333 `generateMutantSpatialWeighted` methods do the same but for spatially structured popula-
 334 tions, where the mutant is placed in a single patch chosen with probability proportional to
 335 the patch population sizes (summed across all resident species).

336 Apart from the four built-in methods listed, there is also a fifth one: `noMutation` simply
 337 returns the supplied community unchanged (useful for testing purposes, or for running purely
 338 ecological dynamics). Otherwise, new mutation methods can be implemented by the user and
 339 substituted into `EcoEvoConfig.mutationGenerator` without any further changes.

2.6 Visualization, I/O, and tests

Plotting recipes live in two extensions, loaded automatically when `Plots` or `GLMakie` is available. `plotEvo` produces a one-trait timeline colored by population density; `plotEvoTwoTrait` and `plotEvoTwoTraitInteractive` provide static and interactive views of two-dimensional trait spaces. `historyToTable` converts an `EvoHistory` into a long-format table suitable for export via `CSV.jl`. The table's columns are: `mutNo` (mutation event, from zero up), `time` (the amount of integration time that passed since the simulation was initiated), `species` (the species index, going from 1 up), `popsiz_1`, `popsiz_2`, ... (population size of the species in the given row in the 1st, 2nd, ... stage class—there are as many columns as stage classes), and `trait_1`, `trait_2`, ... (1st, 2nd, ... component of the given species' trait vector, for as many columns as there are trait dimensions). The full source is accompanied by a test suite (under `test/`) that tries out every public function on small reference problems.

To summarize the complete workflow, below is a fully-fledged script simulating the dynamics, visualizing its results, and generating a table from them:

```
using EcoEvoSim, Plots, Random, CSV

b(z) = 1 - sum(z)^2
a(zi, zj) = -1

config = EcoEvoConfig(
    ecoDyn = lotkaVolterra(b, a),
    mutationGenerator = generateMutant(
        invaderPopsiz = 0.001,
        variance = 0.001^2
    ),
    integrationParams = IntegrationParams(
        maxTime = 1e12,
        abstol = 1e-8,
        reltol = 1e-8
    ),
    extThreshold = 0.003
)

# Set random seed for reproducibility:
Random.seed!(54321)

# Initialize with single species at ecological equilibrium:
lineage = Community([1.0], [-0.4], Float64[])
lineage = ecoDyn(lineage, config)

# Evolve for 1500 mutation events:
lineage = evolve(lineage, config, 1500);

# Create and save a visualization of the process:
p = plotEvo(lineage)
savefig(p, "evo-plot.png")
```

```

386
387 # Create a table of the results and save it in CSV format:
388 table = historyToTable(lineage)
389 CSV.write("evo-table.csv", table)

```

2.7 The R interface: **ecoevosimr**

For users whose primary workflow is in R, the companion package **ecoevosimr** wraps every public function of **EcoEvoSim** and re-exposes it as a regular R function. Internally it relies on **JuliaConnectoR** (Lenz et al. 2022) to start a Julia process in the background, ship objects across the language boundary, and dispatch calls to the underlying Julia code. From the user's point of view, this layer is invisible: communities, configurations, and histories are R objects (S3 classes `ees_community`, `ees_config`, `ees_history`, etc.), constructed and manipulated with normal R syntax.

A full simulation, implementing the same model and parameterization as the Julia script above, looks like this in R:

```

400 library(ecoevosimr)
401 library(tidyverse)
402
403 # Replace path below to the Julia package as needed:
404 eesSetup("~/projects/github/EcoEvoSim") # Invoke once per session
405
406 config <- ecoEvoConfig(
407   lotkaVolterra(
408     growthFn = "z -> 1 - sum(z)^2",
409     kernelFn = "(zi, zj) -> -1"
410   ),
411   mutantGenerator("weighted", invaderPopsize = 0.001, variance = 0.001^2),
412   integrationParams(1e12, abstol = 1e-8, reltol = 1e-8),
413   extThreshold = 0.003
414 )
415
416 initCommunity <- ecoDyn(community(1.0, -0.4), config)
417 lineage <- evolve(initCommunity, config, nSteps = 1500L, seed = 54321L)
418
419 # Convert output to a tibble, then visualize using ggplot2:
420 lineage |>
421   historyToTibble() |>
422   ggplot(aes(x = trait_1, y = mutNo, color = popsize_1)) +
423   geom_point(alpha = 0.5) +
424   scale_size_area(max_size = 2) +
425   labs(x = "Trait value", y = "Mutation event", color = "Pop. size") +
426   scale_color_gradient(low = "gray90", high = "navy") +
427   theme_bw()

```

Three design choices keep the wrapper light and fast. First, model equations are passed as Julia code strings ("z -> 1 - sum(z.^2) / 0.5^2") rather than R closures. The strings

are parsed and JIT-compiled by Julia once per session, so the inner loop of the ODE solver never has to cross the R–Julia boundary (the cost of which would otherwise dominate the simulation). Second, simulation objects are kept Julia-side and accessed through opaque R proxies, so large histories are not serialized back and forth between languages until the user explicitly asks for them. For example, when running `evolve`, the result is a pure Julia object wrapped in an R container that is an S3 object. Third, conversion to R is deliberately confined to a single function, `historyToTibble`, which materializes the history as a long-format tibble with one row per (mutation event, species) combination and columns for population size and trait values. From there, one can rely on, e.g., the `tidyverse` for further data processing and visualization, as we have done above.

Finally, two further functions aid the execution of arbitrary Julia code, to handle cases that the wrapper does not yet cover. The first is `defineJuliaFn` for defining Julia functions. For example, `defineJuliaFn("ss(z) = sum(z.^2)")` defines a sum-of-squares function that can then be used in a Julia code string (such as `"z -> 1 - ss(z) / 0.5^2"`). The second is `juliaExec` which will evaluate any Julia code—this could be useful for defining constants or for loading custom packages; e.g., `juliaExec("using CSV")` will load the `CSV.jl` package in Julia. The R application programming interface is documented in `man/` and summarized in the package’s README.

3 A worked example: competition-proliferation tradeoff in tumor clone evolution

3.1 The model and its implementation

We now showcase how the package can be used through an example. The example is inspired by cancer biology and how several, ecologically distinct tumor clones can arise through evolutionary processes (Lloyd et al. 2016; Kotler and Brown 2020). We focus on one particular tradeoff that may occur in tumors: a competition-proliferation tradeoff—better known in the ecological literature as either a competition-colonization (Tilman 1994; Kinzig et al. 1999; Miller and Max 2025) or competition-mortality (Adler and Mosquera 2000; D’Andrea et al. 2013) tradeoff. The point of this tradeoff is that the speed of cancer cell proliferation is an evolving trait, and cells that prioritize rapid division incur metabolic costs in the form of reduced oxidative phosphorylation efficiency (Aktipis et al. 2013) which renders them inferior competitors for limited resources (Hanahan and Weinberg 2011). The `evolving trait  $z$` is the `proliferation capacity` that determines, through a growth function, how well a species can grow in the absence of competition; in return, it will be less competitive (so low values of z correspond to good competitors and slow growers, while a high z means a weak competitor that proliferates fast).

Mathematically, the model can be cast in the form of Eqn 1. What makes it a competition-proliferation model is choosing $b(z)$ as a monotonically increasing function of z , and choosing $a(z_i, z_j)$ to encode a hierarchy: a slower-growing clone with lower z should be competitively superior to a faster-growing one with higher z . One possible implementation proceeds by first defining the `sigmoid` function

$$Q(z) = \frac{\tanh(z) + 1}{2}, \quad (4)$$

which is 0 for $z \ll 0$, 1 for $z \gg 0$, and has its inflection point at $z = 0$. The `intrinsic growth function  $b(z)$` is expressed in terms of this function:

$$b(z) = Q\left(\frac{z - z_0}{\eta}\right) - Q\left(-\frac{z_0}{\eta}\right), \quad (5)$$

where z_0 is the point at which the curve transitions from low to high values, and η measures the smoothness of this transition. The second, z -independent negative term above simply ensures $b(0) = 0$; i.e., that a clone with trait $z = 0$ has zero growth (Figure 1, left). Since this also means that $b(z < 0) < 0$, clones with negative phenotypes cannot grow at all even under ideal, competition-free conditions. Consequently $z = 0$ is the most competitive phenotype in this parameterization that could in principle arise, making it a natural point of origin for the model. In turn, the interaction kernel is written

$$a(z, z') = -Q\left(\frac{z - z'}{\sigma}\right), \quad (6)$$

where σ measures how different two phenotypes must be for one to have substantial competitive superiority over the other (Figure 1, right). The negative sign expresses the fact that interactions are competitive, with phenotypes always hindering one another's growth. The magnitude of competition is given by the competition kernel $-a(z, z')$.

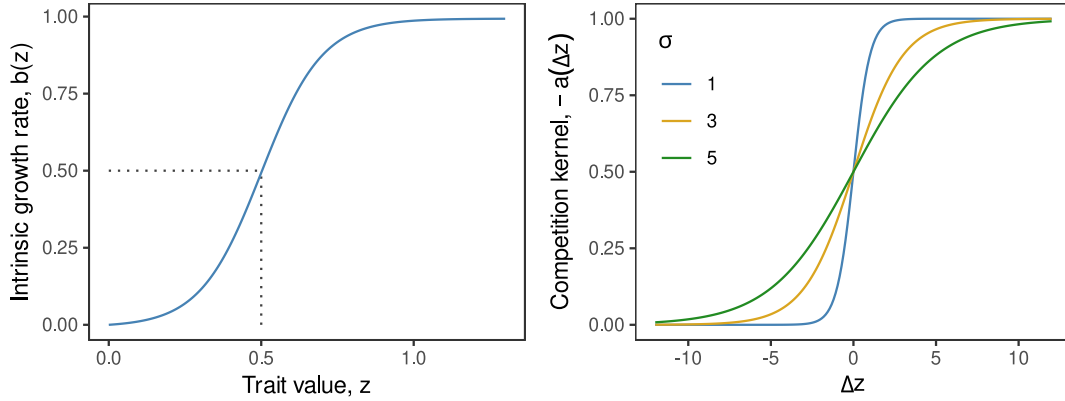


Figure 1: Shape of the intrinsic growth function $b(z)$ (Eqn 5; left panel) and the competition kernel (the negative of the interaction kernel in Eqn 6; right panel). The latter is shown for various values of σ (colors). The transition width σ is proportional to the distance over which the function transitions from values near zero to values near one.

The script below implements the model in `EcoEvoSim`, configures the simulator, and runs 10 000 mutation events:

```
using EcoEvoSim

Q(x) = (tanh(x) + 1) / 2
growthFn(z) = Q((sum(z) - 0.5) / 0.2) - Q(-0.5 / 0.2)
kernelFn(zi, zj) = -Q((sum(zi) - sum(zj)) / 0.13)

config = EcoEvoConfig(
    ecoDyn = lotkaVolterra(growthFn, kernelFn),
    mutationGenerator = generateMutantWeighted(
        invaderPopsiz = 0.001,
        variance = 0.001^2
    ),
)
```

```

502     integrationParams = IntegrationParams(
503         maxTime = Inf,
504         algorithm = DynamicSS(),
505         abstol = 1e-14,
506         reltol = 1e-8
507     ),
508     extThreshold = 0.003
509 )
510
511 initCommunity = ecoDyn(Community([1.0], [0.3]), config)
512 lineage = evolve(initCommunity, config, 10_000)
513 lineageTab = historyToTable(lineage)

```

514 The choice `DynamicSS()` for the ODE integration algorithm causes the ecological phase to
515 integrate to steady state at every mutation event; this is desirable when one is interested in
516 the limiting community structure rather than transient dynamics. `generateMutantWeighted`
517 ensures that abundant clones are more likely to give rise to the next mutant than rare ones,
518 mimicking the per-capita nature of mutation.

519 The same simulation can be expressed in R via `ecoevosimr`. This is in fact how the
520 trajectory shown in Figure 2 was generated, allowing both the simulation and its downstream
521 analysis (community matrix, inverse, plotting) to live in a single R script:

```

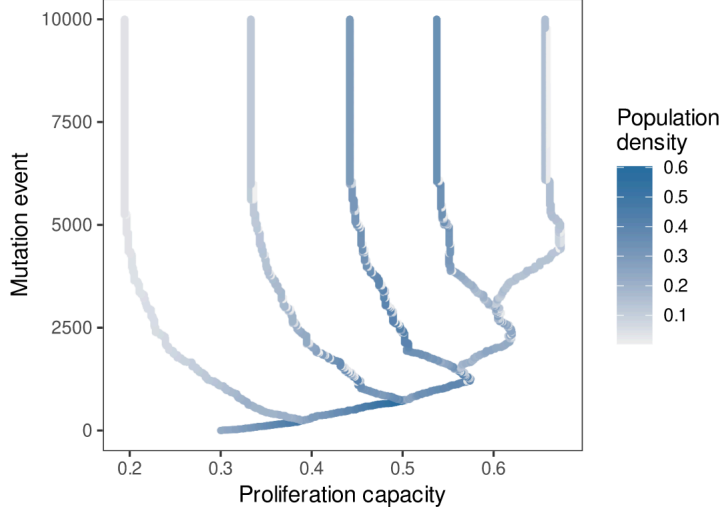
522 library(ecoevosimr)
523
524 # Replace path below to the Julia package as needed:
525 eesSetup("~/projects/github/EcoEvoSim") # Invoke once per session
526
527 defineJuliaFn("Q(z) = (tanh(z) + 1) / 2")
528 defineJuliaFn("growthFn(z) = Q((sum(z) - 0.5) / 0.2) - Q(-0.5 / 0.2)")
529 defineJuliaFn("kernelFn(zi, zj) = -Q((sum(zi) - sum(zj)) / 0.13)")
530
531 config <- ecoEvoConfig(
532     ecoDyn = lotkaVolterra("growthFn", "kernelFn"),
533     mutationGenerator = mutantGenerator(
534         "weighted",
535         invaderPopsize = 0.001,
536         variance = 0.001^2
537     ),
538     integrationParams = integrationParams(
539         maxTime = Inf,
540         algorithm = "DynamicSS()",
541         abstol = 1e-14,
542         reltol = 1e-8
543     ),
544     extThreshold = 0.003
545 )
546

```

```

547 initCommunity <- ecoDyn(community(1.0, 0.3), config)
548 lineage <- evolve(initCommunity, config, nSteps = 10000L, seed = 54321L)
549 lineageTab <- historyToTibble(lineage)

```



550

551 **Figure 2:** Eco-evolutionary trajectory of the competition-mortality model, with $b(z)$ and
552 $a(z, z')$ given by Eqn 5 and Eqn 6 ($z_0 = 0.5$, $\eta = 0.2$, $\sigma = 0.13$). A single ancestor at $z =$
553 0.3 diversifies, over 10 000 mutation events, into a stable community of five hierarchically
554 organized clones.

555 3.2 Community structure and cascade analysis

556 The endpoint of the simulation is a stable community of five clones spread along the prolif-
557 eration axis. To analyze this community, we compute the community matrix $\mathbf{M}$ (Novak et al.
558 2016), which is the Jacobian of Eqn 1 (given by Eqn 2) evaluated at the final eco-evolutionarily
559 stable community state (Edwards et al. 2018). Let n_i^* denote the equilibrium population density
560 of clone i . Then the entries M_{ij} of the community matrix are

$$M_{ij} = J_{ij}|_{n_i=n_i^*} = \underbrace{\delta_{ij} \left(b(z_i) + \sum_k a(z_i, z_k) n_k^* \right)}_0 + n_i^* a(z_i, z_j) = n_i^* a(z_i, z_j). \quad (7)$$

561 (The first term was the per capita growth rate of each species at equilibrium, which is zero for
562 any persisting species by definition, so that whole term cancels.) The resulting matrix, along
563 with its inverse, is shown in Figure 3.

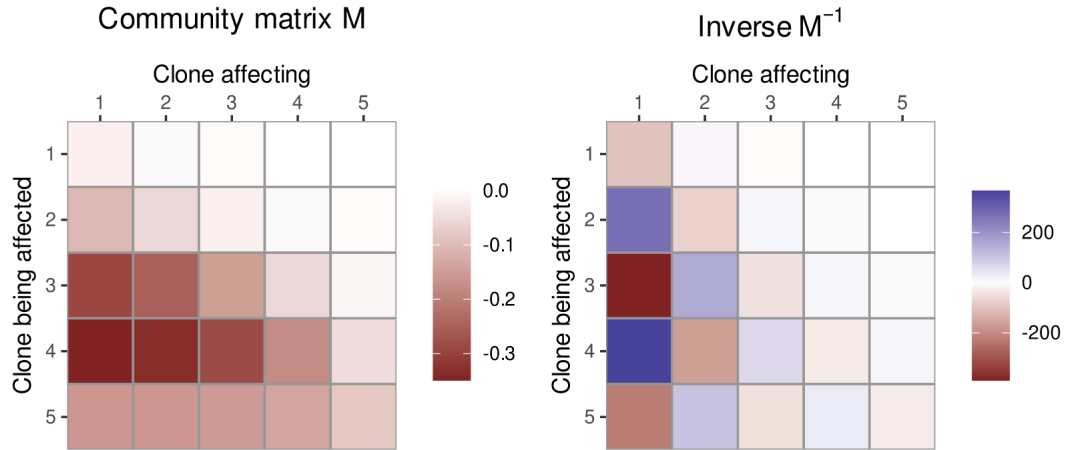


Figure 3: (left) Community matrix $\mathbf{M}$ at the eco-evolutionary endpoint of Figure 2. The matrix is nearly lower-triangular, the signature of a competitive hierarchy. (right) Inverse community matrix $\mathbf{M}^{-1}$, whose (i, j) entry quantifies the long-term effect on clone i of a sustained press perturbation to clone j . Alternating-sign rows indicate cascading indirect effects.

Two features of Figure 3 are noteworthy. First, $\mathbf{M}$ is nearly lower-triangular: the slow-growing, competitively dominant clone (clone 1) suppresses everyone below it, but is itself almost untouched by them. This is the asymmetric structure that the competition-mortality tradeoff is designed to produce, and it emerges spontaneously here from the eco-evolutionary process rather than being imposed by hand.

Second, $\mathbf{M}^{-1}$, whose (i, j) th entry gives the long-term sensitivity of clone i to a sustained press perturbation of clone j (Bender et al. 1984; Yodzis 1988; Novak and Wootton 2010), shows the alternating-sign “checkerboard” pattern characteristic of hierarchical competition. A press that increases the abundance of the top competitor causes a strong decline in clones 2–4; a press in the opposite direction (decreasing it) releases them.

To find the perturbation that the community amplifies most strongly, we compute the singular value decomposition of $\mathbf{M}^{-1}$. The right singular vector associated with the largest singular value identifies the direction in perturbation space that produces the largest community-wide response. For the present community this leading direction is dominated by the top competitor, confirming quantitatively that perturbing clone 1 yields the greatest leverage. Importantly, the response vector spreads the consequences of that perturbation across the rest of the community, so that a single targeted intervention propagates through the hierarchy.

4 Discussion

In this work we introduced the Julia package `EcoEvoSim`, which aims to make eco-evolutionary simulation as routine in cancer biology and community ecology as ODE integration already is. Its small, composable type system and factory-based model interface let users specify what is biologically interesting about their system (the dynamical equations, the tradeoffs, the relevant trait dimensions) without having to re-derive the simulation machinery for every new question. The companion R package `ecoevosimr` makes the same machinery available to users who prefer to work in R, with no loss of performance and full access to the tidyverse for downstream analysis. Applied to a cancer-relevant competition-mortality model, the package recovers the hierarchical clonal structure expected on theoretical grounds and exposes, via

the inverse community matrix, the cascading indirect effects that motivate a new therapeutic strategy: ecological cascade therapy.

Three points are worth emphasizing about the worked example. (i) The hierarchical structure of the final community was not built into the simulation; it emerged from the iterated interplay of mutation, selection, and the postulated tradeoff. (ii) The cascade revealed by $\mathbf{M}^{-1}$ is *not* visible in $\mathbf{M}$ itself: pairwise interactions only tell us who suppresses whom directly, while the inverse tells us how the network reroutes those interactions when the system is perturbed. (iii) Both kinds of information—the assembly of the community and the community matrix at its endpoint—fall out of a single `EcoEvoSim` configuration of fewer than twenty lines, with no special tooling.

The cascade structure suggests a therapeutic principle that complements adaptive (Gatenby et al. 2009) and double-bind (Basanta et al. 2012) approaches. Rather than directly attacking the most resistant phenotype—which is often the most difficult clone to drug, by virtue of being slow-growing, quiescent, or expressing few targetable markers—one can target an *accessible* clone whose removal forces a cascade through the hierarchy. In our example the most accessible clone (the top competitor) is also the most influential, but in general the two need not coincide; the SVD analysis identifies the most leveraged accessible perturbation. We call this strategy **ecological cascade therapy**. `EcoEvoSim` is well suited to designing and stress-testing such strategies, because the same configuration that generates the resident community can be re-used to simulate its response to a wide range of perturbations.

`EcoEvoSim` is deterministic and operates in the rare-mutation, separated-timescales limit of adaptive dynamics (Metz et al. 1996; Geritz et al. 1998). This is appropriate when the ecological time scale is much shorter than the mutational one, as is often the case for solid tumors of moderate size, but it abstracts away the demographic stochasticity that governs the very early fate of a single mutant cell. The assumption of fixed, heritable trait values likewise abstracts away phenotypic plasticity; bridging these limits is an obvious next step, and the modular factory-function design makes it straightforward to plug in alternative mutation generators (e.g., establishment based on a branching process) or alternative ecological dynamics (e.g., piecewise-deterministic Markov processes).

Beyond cancer, the same engine has already been used to simulate (i) competition in two-dimensional trait spaces, (ii) two-patch source–sink dynamics with and without explicit resource flow, and (iii) consumer–resource dynamics on a radial transect with patch-specific immune pressure (the food-safety tradeoff). Because the simulator is agnostic to the meaning of `Species`, `Community`, and `Phenotype`, models of microbial communities, plant guilds, and host–pathogen systems all fit naturally within the same framework.

Data and code availability

The `EcoEvoSim` package is hosted at <https://github.com/dysordys/EcoEvoSim> under the MIT licence. The R wrapper, `ecoevosimr`, is hosted at <https://github.com/dysordys/ecoevosimr>, also under the MIT licence. All simulation scripts, model definitions, and analysis code used in this paper are in the companion repository <https://github.com/dysordys/cancer-ecoevo>.

Acknowledgments

Bibliography

- Adler, F. R., and J. Mosquera. 2000. Is space necessary? Interference competition and limits to biodiversity. *Ecology* 81:3226–3232.
- Aktipis, C. A., and R. M. Nesse. 2013. Evolutionary foundations for cancer biology. *Evolutionary Applications* 6:144–159.
- Aktipis, C. A., A. M. Boddy, R. A. Gatenby, J. S. Brown, and C. C. Maley. 2013. Life history trade-offs in cancer evolution. *Nature Reviews Cancer* 13:883–892.
- Barabás, G., and R. D'Andrea. 2016. The effect of intraspecific variation and heritability on community pattern and robustness. *Ecology Letters* 19:977–986.
- Barabás, G., C. Parent, A. Kraemer, F. Van de Perre, and F. De Laender. 2022. The evolution of trait variance creates a tension between species diversity and functional diversity. *Nature Communications* 13:1–10.
- Barton, N. H., A. M. Etheridge, and A. Véber. 2017. The infinitesimal model: Definition, derivation, and implications. *Theoretical Population Biology* 118:50–73.
- Baruah, G., G. Barabás, and R. John. 2025. When Do Trait-Based Higher Order Interactions and Individual Variation Promote Robust Species Coexistence?. *Ecology and Evolution* 15:e71336.
- Basanta, D., R. A. Gatenby, and A. R. A. Anderson. 2012. Exploiting Evolution To Treat Drug Resistance: Combination Therapy and the Double Bind. *Molecular Pharmaceutics* 9:914–921.
- Bender, E. A., T. J. Case, and M. E. Gilpin. 1984. Perturbation Experiments in Community Ecology: Theory and Practice. *Ecology* 65:1–13.
- Bezanson, J., A. Edelman, S. Karpinski, and V. B. Shah. 2017. Julia: A Fresh Approach to Numerical Computing. *SIAM Review* 59:65–98.
- Brännström, Å., J. Johansson, and N. Von Festenberg. 2013. The hitchhiker's guide to adaptive dynamics. *Games* 4:304–328.
- Case, T. J. 2000. *An Illustrated Guide to Theoretical Ecology*. Oxford University Press, New York, USA.
- Chesson, P., and N. Huntly. 1997. The roles of harsh and fluctuating conditions in the dynamics of ecological communities. *American Naturalist* 150:519–553.
- D'Andrea, R., G. Barabás, and A. Ostling. 2013. Revising the Tolerance-Fecundity Trade-Off; or, On the Consequences of Discontinuous Resource Use for Limiting Similarity, Species Diversity, and Trait Dispersion. *American Naturalist* 181:E91–101.
- Dalui, D., and J. P. DeLong. 2025. Modeling the Tangled Bank: A Users Guide to Gillespie Eco-evolutionary Models using the Julia Programming Language.
- Dieckmann, U., and R. Law. 1996. The dynamical theory of coevolution: a derivation from stochastic ecological processes. *Journal of Mathematical Biology* 34:579–612.
- Dolson, E., and C. Ofria. 2021. Digital Evolution for Ecology Research: A Review. *Frontiers in Ecology and Evolution* 9:750779.
- Edwards, K. F., C. T. Kremer, E. T. Miller, M. M. Osmond, E. Litchman, and C. A. Klausmeier. 2018. Evolutionarily stable communities: a framework for understanding the role of trait evolution in the maintenance of diversity. *Ecology Letters* 21:1853–1868.
- Gatenby, R. A., and R. J. Gillies. 2008. A microenvironmental model of carcinogenesis. *Nature Reviews Cancer* 8:56–61.

Gatenby, R. A., A. S. Silva, R. J. Gillies, and B. R. Frieden. 2009. Adaptive Therapy. *Cancer Research* 69:4894–4903.

Geritz, S. A. H., É. Kisdi, G. Meszéna, and J. A. J. Metz. 1998. Evolutionarily singular strategies and the adaptive growth and branching of the evolutionary tree. *Evolutionary Ecology* 12:35–57.

Govaert, L., E. A. Fronhofer, S. Lion, C. Eizaguirre, D. Bonte, M. Egas, A. P. Hendry, et al. 2019. Eco-evolutionary feedbacks—Theoretical models and perspectives. *Functional Ecology* 33:13–30.

Greaves, M., and C. C. Maley. 2012. Clonal evolution in cancer. *Nature* 481:306–313.

Hagen, O., B. Flück, F. Fopp, J. S. Cabral, F. Hartig, M. Pontarp, T. F. Rangel, et al. 2021. gen3sis: A general engine for eco-evolutionary simulations of the processes that shape Earth's biodiversity. *PLOS Biology* 19:e3001340.

Hanahan, D., and R. A. Weinberg. 2011. Hallmarks of cancer: the next generation. *Cell* 144:646–674.

Kinzig, A. P., S. A. Levin, J. Dushoff, and S. W. Pacala. 1999. Limiting similarity, species packing, and system stability for hierarchical competition-colonization models. *American Naturalist* 153:371–383.

Klausmeier, C. A. 2024. EcoEvo: Analyze species- and trait-based ecological and eco-evolutionary models, including Adaptive Dynamics, in Mathematica.

Klausmeier, C. A., C. T. Kremer, and T. Koffel. 2020. Trait-based ecological and eco-evolutionary theory. (K. S. McCann & G. Gellner, eds.) *Theoretical Ecology: Concepts and Applications*. Oxford University Press.

Korolev, K. S., J. B. Xavier, and J. Gore. 2014. Turning ecology and evolution against cancer. *Nature Reviews Cancer* 14:371–380.

Kotler, B. P., and J. S. Brown. 2020. Cancer Community Ecology. *Cancer Control* 27:1–11.

Lenz, S., M. Hackenberg, and H. Binder. 2022. The JuliaConnectoR: A Functionally-Oriented Interface for Integrating Julia in R. *Journal of Statistical Software* 101:1–24.

Levins, R. 1968. *Evolution in changing environments*. Princeton University Press, Princeton, USA.

Liao, J., G. Barabási, and D. Bearup. 2022. Competition–colonization dynamics and multimodality in diversity–disturbance relationships. *Ecology* 103:e3672.

Lloyd, M. C., J. J. Cunningham, M. M. Bui, R. J. Gillies, J. S. Brown, and R. A. Gatenby. 2016. Darwinian dynamics of intratumoral heterogeneity: not solely random mutations but also variable environmental selection forces. *Cancer Research* 76:3136–3144.

MacArthur, R. H., and R. Levins. 1967. Limiting similarity, convergence, and divergence of coexisting species. *American Naturalist* 101:377–385.

Marusyk, A., V. Almendro, and K. Polyak. 2012. Intra-tumour heterogeneity: a looking glass for cancer?. *Nature Reviews Cancer* 12:323–334.

McGranahan, N., and C. Swanton. 2017. Clonal Heterogeneity and Tumor Evolution: Past, Present, and the Future. *Cell* 168.

Merlo, L. M. F., J. W. Pepper, B. J. Reid, and C. C. Maley. 2006. Cancer as an evolutionary and ecological process. *Nature Reviews Cancer* 6:924–935.

Meszéna, G. 2005. Adaptive dynamics: the continuity argument. *Journal of Evolutionary Biology* 18:1182–1185.

Meszéna, G., I. Czibula, and S. Geritz. 1997. Adaptive dynamics in a 2-patch environment: a toy model for allopatric and parapatric speciation. *Journal of Biological Systems* 5:265–284.

Meszéna, G., M. Gyllenberg, F. J. Jacobs, and J. A. J. Metz. 2005. Link between population dynamics and dynamics of Darwinian evolution. *Physical Review Letters* 95:78105.

Metz, J. A. J., and S. A. H. Geritz. 2016. Frequency dependence 3.0: an attempt at codifying the evolutionary ecology perspective. *Journal of Mathematical Biology* 72:1011–1037.

Metz, J. A. J., S. A. H. Geritz, G. Meszéna, F. J. A. Jacobs, and J. S. Van Heerwaarden. 1996. Adaptive dynamics: a geometrical study of the consequences of nearly faithful reproduction. (S. J. van Strien & S. M. Verduyn Lunel, eds.) *Stochastic and Spatial Structures of Dynamical Systems*. North-Holland, Amsterdam.

Miller, Z. R., and D. Max. 2025. Multispecies coexistence emerges from pairwise exclusions in communities with competitive hierarchy. *Ecology Letters* 28:e70206.

Mullon, C., and L. Lehmann. 2019. An evolutionary quantitative genetics model for phenotypic (co)variance under limited dispersal, with an application to socially synergistic traits. *Evolution* 73:1695–1728.

Novak, M., and J. T. Wootton. 2010. Using experimental indices to quantify the strength of species interactions. *Oikos* 119:1057–1063.

Novak, M., J. Yeakel, A. E. Noble, D. F. Doak, M. Emmerson, J. A. Estes, U. Jacob, et al. 2016. Characterizing species interactions to understand press perturbations: what is the community matrix?. *Annual Review of Ecology and Systematics* 47:409–432.

Nowell, P. C. 1976. The clonal evolution of tumor cell populations. *Science* 194:23–28.

Ofria, C. A., and C. O. Wilke. 2004. Avida: a software platform for research in computational evolutionary biology. *Artificial Life* 10:191–229.

Pastore, A. I., G. Barabás, M. D. Bimler, M. M. Mayfield, and T. E. Miller. 2021. The evolution of niche overlap and competitive differences. *Nature Ecology & Evolution* 5:330–337.

Pásztor, L., Z. Botta-Dukát, G. Magyar, T. Czárán, and G. Meszéna. 2016. Theory-based ecology: a Darwinian approach. Oxford University Press, Oxford, UK.

Rackauckas, C., and Q. Nie. 2017. DifferentialEquations.jl – A Performant and Feature-Rich Ecosystem for Solving Differential Equations in Julia. *Journal of Open Research Software* 5:15.

Roughgarden, J. 1979. *Theory of Population Genetics and Evolutionary Ecology: an Introduction*. Macmillan, New York, USA.

Schreiber, S. J., M. Benaïm, and K. A. S. Atchadé. 2011. Persistence in fluctuating environments. *Journal of Mathematical Biology* 62:655–683.

Slatkin, M. 1979. Frequency- and density-dependent selection on a quantitative character. *Genetics* 93:755–771.

Slatkin, M. 1980. Ecological character displacement. *Ecology* 6:163–177.

Szilágyi, A., and G. Meszéna. 2009. Two-patch model of spatial niche segregation. *Evolutionary Ecology* 23:187–205.

Taper, M. L., and T. J. Case. 1985. Quantitative genetic models for the coevolution of character displacement. *Ecology* 66:355–371.

Tilman, D. 1994. Competition and biodiversity in spatially structured habitats. *Ecology* 75:2–16.

Vasseur, D. A., P. Amarasekare, V. H. W. Rudolf, and J. M. Levine. 2011. Eco-evolutionary dynamics enable coexistence via neighbor-dependent selection. *American Naturalist* 178:E96–E109.

Wickham, H. 2016. *ggplot2: Elegant Graphics for Data Analysis*. Springer International Publishing.

771 Wickham, H., M. Averick, J. Bryan, W. Chang, L. D. McGowan, R. François, G. Golemund, et
772 al. 2019. Welcome to the tidyverse. *Journal of Open Source Software* 4:1686.
773 Yodzis, P. 1988. The indeterminacy of ecological interactions as perceived through perturba-
774 tion experiments. *Ecology* 69:508–515.